# Module 2

## Operating System Principles

## Lecture 2

# Protection: Modes – User Mode

- The CPU is spending time in two very distinct modes: User Mode and Kernel Mode.
- **User Mode:**
  - It is a restricted mode that limits the software's access to system resources.
  - Code running in user mode must delegate to system APIs to access hardware or memory.
  - When a user-mode application is started, Windows creates a process for the application.
    - The process provides the application with a private virtual address space and a private handle table. Because an application's virtual address space is private, one application can't alter data that belongs to another application. Each application runs in isolation, and if an application crashes, the crash is limited to that one application.

# Protection: Modes – Kernel Mode

- **Kernel Mode:**
  - It is a privileged mode that allows software to access system resources and perform privileged operations.
  - The executing code has complete and unrestricted access to the underlying hardware.
  - It can execute any CPU instruction and reference any memory address.
  - All code that runs in kernel mode shares a single virtual address space.
    - Therefore, a kernel-mode driver isn't isolated from other drivers and the operating system itself. If a kernel-mode driver accidentally writes to the wrong virtual address, data that belongs to the operating system or another driver could be compromised. If a kernel-mode driver crashes, the entire operating system crashes.
- If an attempt is made to execute a privileged instruction in user mode, the hardware does not execute the instruction but rather treats it as illegal and traps it to the operating system

# Protection: Modes – Kernel Mode

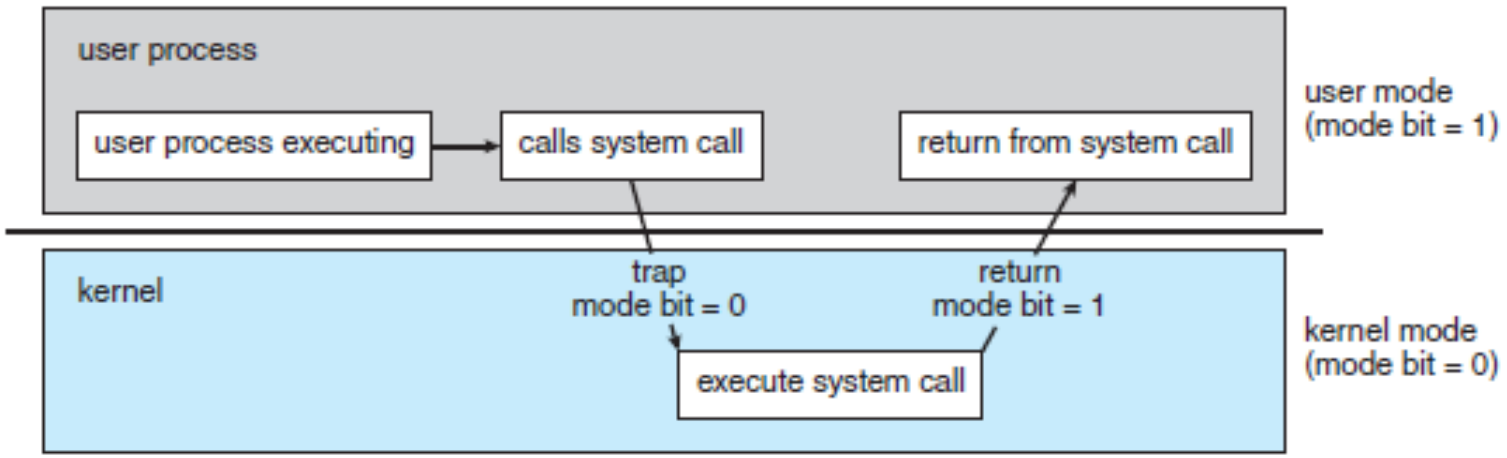



**Figure:** Transition from user to kernel mode

- At system boot time, the hardware starts in kernel mode. The operating system is then loaded and starts user applications in user mode.
- Whenever a trap or interrupt occurs, the hardware switches from user mode to kernel mode
- Whenever the operating system gains control of the computer, it is in kernel mode. The system always switches to user mode before passing control to a user program.
- The dual mode of operation provides us with the means for protecting the operating system from errant users.

# Interrupts

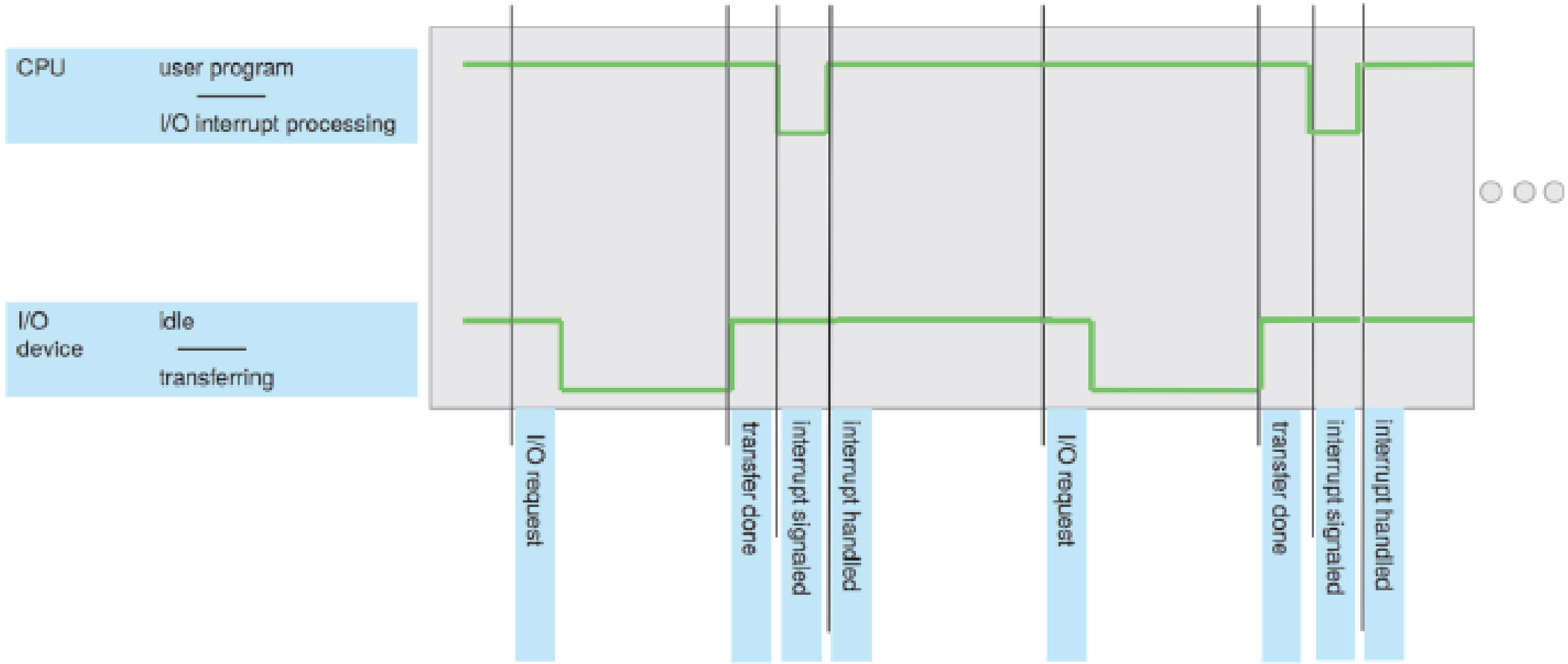
- An interrupt is a signal emitted by hardware or software when a process or an event needs immediate attention. It alerts the processor to a high-priority process requiring interruption of the current working process.

- An interrupt in an operating system is a signal that prompts the OS to temporarily halt its current activities and execute a specific function, often referred to as interrupt service routine (ISR).

# Interrupts

- Hardware may trigger an interrupt at any time by sending a signal to the CPU through the system bus.
- When the CPU is interrupted, it stops what it is doing and immediately transfers execution to a fixed location.
- The fixed location usually contains the starting address where the service routine for the interrupt is located.
- The interrupt service routine executes; on completion, the CPU resumes the interrupted computation.
- Interrupts must be handled quickly, as they occur very frequently.

# Figure: Interrupt timeline for a single program doing output.



# Interrupt Handling Process

- Interrupt Request: The hardware device sends an interrupt request (IRQ) to the CPU.
- Acknowledgment: The CPU acknowledges the interrupt and temporarily pauses its current execution.
- Interrupt Vector: The CPU uses an interrupt vector to locate the appropriate interrupt handler. This vector is essentially *a table of pointers to interrupt service routines*.
- Interrupt Service Routine (ISR): The CPU executes the ISR to handle the interrupt. This may involve reading data from a device, processing input, or handling an error.
- Resume Execution: After the ISR completes, the CPU resumes its previous activity from where it was interrupted.

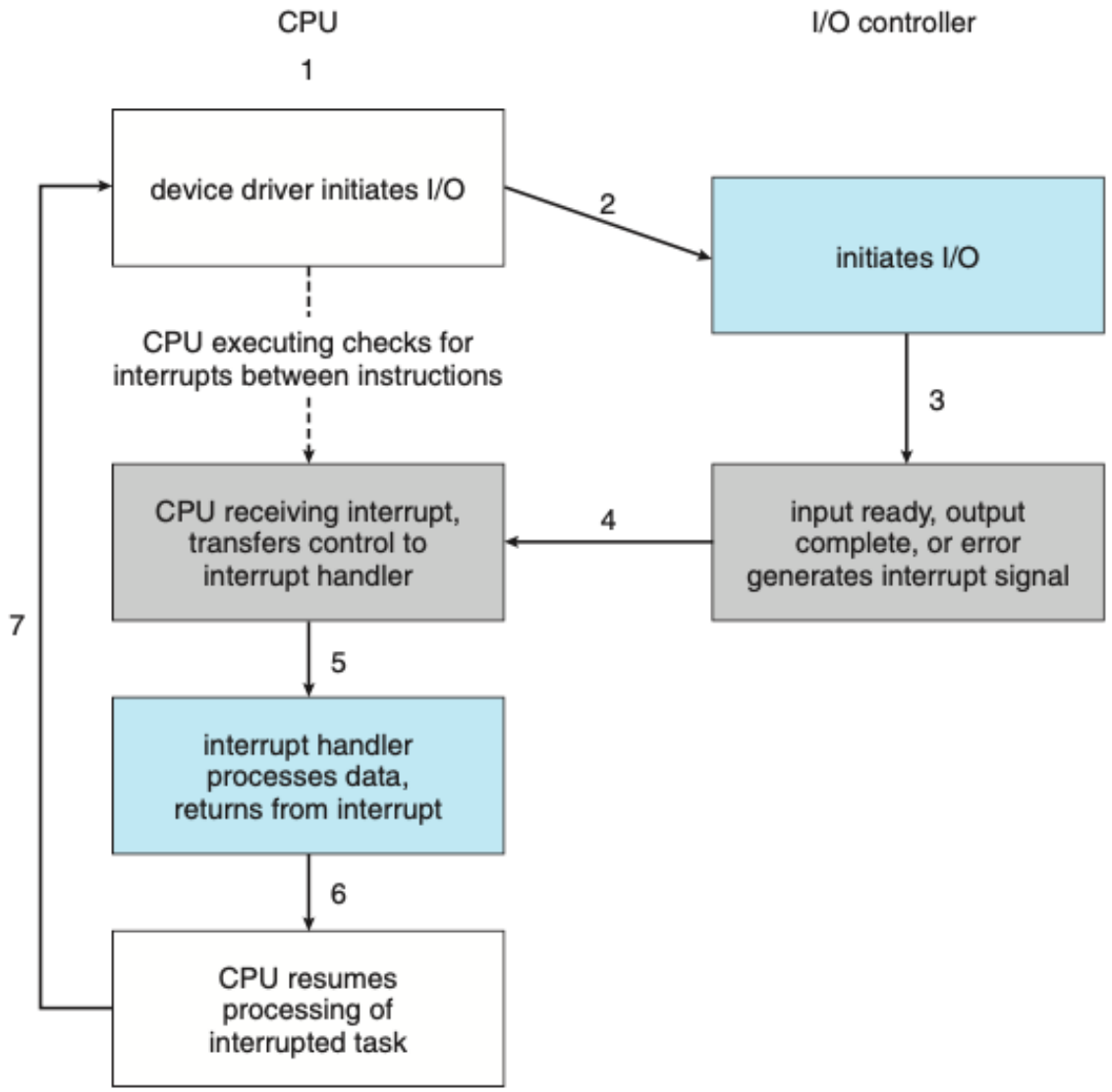
# Interrupt Implementation

- The CPU hardware has a wire called the Interrupt-Request Line that the CPU senses after executing every instruction.
- When the CPU detects that a controller has asserted a signal on the interrupt-request line, it reads the interrupt number and jumps to the interrupt-handler routine by using that interrupt number as an index into the interrupt vector.
- It then starts execution at the address associated with that index.

# Interrupt Implementation

- The interrupt handler saves any state it will be changing during its operation, performs the necessary processing, and executes a return from interrupt instruction to return the CPU to the execution state prior to the interrupt.

# Interrupt Driven I/O Cycle



# Features needed for modern day Interrupt handlers

- Ability to defer interrupt handling during critical processing
- An efficient way to dispatch to the proper interrupt handler for a device.
- Have multilevel interrupt handling mechanism to distinguish between high- and low-priority interrupts and can respond with the appropriate degree of urgency.

# Interrupt types

1. Maskable interrupts
2. Non-maskable interrupts

- A Maskable Interrupt is the one that is capable of ignoring/ enabling the instructions of the system's CPU alone.
- The type of interrupt that the instructions of a system's CPU can easily ignore or disable is known as Non-Maskable Interrupt. Such a type of interrupt usually comes into play when the response time is very critical.
- **Interrupt priority levels**